

LAPORAN PRAKTIKUM MODUL 6

Pemrograman Berorientasi Objek (OOP) pada PHP

Disusun oleh:

Fathir Syahragil Yusuf

707012500050

SIKC 49-04

Mata Kuliah: Pemrograman Web

Universitas Telkom

2026

1.1 TUJUAN

1. Memahami konsep dasar Pemrograman Berorientasi Objek (OOP) dalam PHP.
2. Membuat dan menggunakan class serta object dalam PHP.
3. Memahami konsep constructor dan destructor.
4. Menerapkan konsep encapsulation dan inheritance pada studi kasus CRUD LifeTrack.

1.2 ALAT DAN BAHAN

5. Komputer/laptop dengan prosesor minimal 1 GHz, RAM 2GB, dan ruang penyimpanan 1GB.
6. XAMPP (Apache + MySQL).
7. Teks editor (VS Code).
8. Browser web (Google Chrome).

1.3 LANGKAH-LANGKAH PRAKTIKUM

Pada modul ini, project LifeTrack dari Modul 5 refactor agar seluruh logika menggunakan prinsip OOP. Tampilan (HTML/CSS) tidak diubah, hanya bagian PHP yang dipindahkan ke dalam class.

1. Membuat folder classes

Membuat folder classes untuk menampung semua class OOP, supaya struktur project lebih rapi dan terpisah dari file view.

2. Class Database (koneksi PDO)

File classes/Database.php berisi class Database dengan properti private (\$host, \$dbname, \$dbuser, \$dbpass, \$pdo). Properti dibuat private sebagai bentuk encapsulation, lalu disediakan method getPdo() untuk mengakses objek PDO. Konstruktor (__construct) otomatis membuat koneksi saat objek dibuat, dan destruktur (__destruct) menutup koneksi saat objek dihapus.

3. Class Model sebagai parent class

File classes/Model.php berisi class Model yang menjadi induk untuk class CRUD. Di dalamnya terdapat method dasar all(), find(\$id), dan delete(\$id). Properti \$pdo dibuat protected supaya bisa diwariskan ke class turunan.

4. Class Pasien (inheritance dari Model)

File classes/Pasien.php berisi class Pasien extends Model. Class ini mewarisi semua method dari Model dan menambahkan method khusus seperti create(), update(), serta meng-override method delete() agar sekaligus menghapus file dokumen di folder uploads/. Method uploadFile() dan hapusFile() dibuat private karena hanya dipakai di dalam class itu sendiri.

5. Class Auth (login, register, session)

File classes/Auth.php berisi class Auth yang menangani register(), login(), check(), user(), dan logout(). Konstruktor otomatis menjalankan session_start() jika session belum aktif.

6. Refactor file view (beranda, login, registrasi)

File beranda.php, login.php, proseslogin.php, prosesregistrasi.php, dan registrasi.php ubah agar menggunakan instance class. HTML dan CSS-nya sama persis dengan Modul 5, hanya logic PHP-nya yang dipindahkan ke pemanggilan method class.

7. Satu tombol Simpan untuk CRUD + Upload

Tombol Simpan tetap satu seperti modul sebelumnya. Saat form di-submit, beranda.php memanggil \$pasien->create() atau \$pasien->update() yang sudah otomatis memproses upload file di dalamnya.

1.4 HASIL DAN DOKUMENTASI

AUTH.JAVA

```
1  <?php
2  // Class Auth - menangani registrasi, login, logout, cek session
3  class Auth {
4      public function __construct() {
5          if (session_status() === PHP_SESSION_NONE) {
6              session_start();
7          }
8      }
9
10     // simpan akun hasil registrasi ke session
11     public function register($data) {
12         $_SESSION["akun"] = [
13             "role" => trim($data["role"]),
14             "nama" => trim($data["nama"]),
15             "email" => trim($data["email"]),
16             "password" => trim($data["password"]),
17         ];
18     }
19
20     // cek email & password, return true kalau cocok
21     public function login($email, $password) {
22         $akun = $_SESSION["akun"] ?? [];
23         if (($akun["email"] ?? "") === $email && ($akun["password"] ?? "") === $password) {
24             $_SESSION["user"] = $akun["nama"];
25             $_SESSION["role"] = $akun["role"];
26             return true;
27         }
28         return false;
29     }
30
31     public function check() {
32         return isset($_SESSION["user"]);
33     }
34
35     public function user() {
36         return $_SESSION["user"] ?? null;
37     }
38
39     public function akun() {
40         return $_SESSION["akun"] ?? [];
41     }
42
43     public function logout() {
44         session_destroy();
45     }
46 }
47 ?>
```

DATABASE.JAVA

```
1 <?php
2 // Class koneksi database (OOP)
3 class Database {
4     private $host = "localhost";
5     private $dbname = "lifetrack";
6     private $dbuser = "root";
7     private $dbpass = "";
8     private $pdo;
9
10    public function __construct() {
11        try {
12            $this->pdo = new PDO(
13                "mysql:host=$this->host;dbname=$this->dbname;charset=utf8mb4",
14                $this->dbuser,
15                $this->dbpass
16            );
17            $this->pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
18            $this->pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
19        } catch (PDOException $e) {
20            die("Koneksi database gagal: " . $e->getMessage());
21        }
22    }
23
24    public function getPdo() {
25        return $this->pdo;
26    }
27 }
28 ?>
```

MODEL.JAVA

```
1 <?php
2 // Parent class Model - jadi induk untuk class CRUD
3 require_once __DIR__ . "/Database.php";
4
5 class Model {
6     protected $pdo;
7     protected $table;
8
9     public function __construct() {
10         $db = new Database();
11         $this->pdo = $db->getPdo();
12     }
13
14     public function all() {
15         $stmt = $this->pdo->query("SELECT * FROM $this->table ORDER BY created_at DESC");
16         return $stmt->fetchAll();
17     }
18
19     public function find($id) {
20         $stmt = $this->pdo->prepare("SELECT * FROM $this->table WHERE id = ?");
21         $stmt->execute([(int)$id]);
22         return $stmt->fetch();
23     }
24
25     public function delete($id) {
26         return $this->pdo->prepare("DELETE FROM $this->table WHERE id = ?")
27             ->execute([(int)$id]);
28     }
29 }
30 ?>
```

PASIENT.JAVA

```
1 <?php
2 // Class Pasien - turunan dari Model (inheritance)
3 require_once __DIR__ . "/Model.php";
4
5 class Pasien extends Model {
6     private $uploadDir = "uploads/";
7     private $allowedExt = ["jpg", "jpeg", "png", "gif", "webp", "pdf"];
8     private $maxSize = 5242880; // 5MB
9
10    public function __construct() {
11        parent::__construct(); // panggil konstruktor parent
12        $this->table = "data_pasien";
13    }
14
15    // proses upload file, return nama file atau null
16    private function uploadFile($file) {
17        if (!isset($file) || $file["error"] !== UPLOAD_ERR_OK) return null;
18
19        if (!is_dir($this->uploadDir)) mkdir($this->uploadDir, 0755, true);
20
21        $ext = strtolower(pathinfo($file["name"], PATHINFO_EXTENSION));
22        if ($file["size"] > $this->maxSize || !in_array($ext, $this->allowedExt)) {
23            return null;
24        }
25
26        $nama = "doc_" . uniqid() . "_" . time() . "." . $ext;
27        if (!move_uploaded_file($file["tmp_name"], $this->uploadDir . $nama)) {
28            return null;
29        }
30        return $nama;
31    }
32
33    // hapus file dokumen dari folder uploads
34    private function hapusFile($namaFile) {
35        if ($namaFile && file_exists($this->uploadDir . $namaFile)) {
36            unlink($this->uploadDir . $namaFile);
37        }
38    }
39}
```

```
1 // CREATE
2 public function create($data, $file = null) {
3     $dokNama = $this->uploadFile($file);
4     $this->pdo->prepare(
5         "INSERT INTO data_pasien (nama, usia, keluhan, dokumen, tanggal) VALUES (?, ?, ?, ?, ?)"
6     );
7     $data["nama"], $data["usia"], $data["keluhan"], $dokNama, date("Y-m-d")
8     $this->pdo->execute($data);
9 }
10
11 // UPDATE
12 public function update($id, $data, $file = null) {
13     $dokNama = $this->uploadFile($file);
14     if ($dokNama) {
15         // hapus file lama dulu
16         $old = $this->find($id);
17         if ($old) $this->hapusFile($old["dokumen"]);
18     }
19
20     $this->pdo->prepare(
21         "UPDATE data_pasien SET nama=?, usia=?, keluhan=?, dokumen=? WHERE id=?"
22     );
23     $data["nama"], $data["usia"], $data["keluhan"], $dokNama, $id
24     $this->pdo->execute($data);
25 }
26
27 // override delete dari parent supaya sekalian hapus file (overriding)
28 public function delete($id) {
29     $row = $this->find($id);
30     if ($row) {
31         $this->hapusFile($row["dokumen"]);
32         return parent::delete($id);
33     }
34     return false;
35 }
```

1.5 ANALISIS DAN PEMBAHASAN

Refactor dari kode prosedural ke OOP membuat struktur project lebih rapi. Logic database dan logic CRUD yang sebelumnya bercampur di dalam beranda.php sekarang

dipisah ke class masing-masing. File view jadi lebih ringkas karena cukup memanggil method class, tidak perlu menulis query SQL panjang lagi.

Tombol Simpan tetap satu, dan proses upload file sudah dipindahkan ke method private `uploadFile()` di class `Pasien`. Method ini melakukan validasi ekstensi (jpg, png, pdf) dan ukuran file (maksimal 5MB) sebelum file dipindahkan ke folder uploads.

1.6 KESIMPULAN

Dengan menerapkan OOP pada project LifeTrack, struktur kode jadi lebih terorganisir, mudah dipelihara, dan mudah dikembangkan. Konsep encapsulation membantu menjaga data agar tidak diakses sembarangan, inheritance memungkinkan pengurangan duplikasi kode lewat parent-child class, dan constructor-destructor memudahkan pengelolaan siklus hidup objek seperti koneksi database. Hasil akhirnya, fitur CRUD + upload file LifeTrack tetap berjalan sama seperti Modul 5, tetapi jauh lebih bersih dan mengikuti prinsip OOP.

1.7 LAMPIRAN

Link GitHub:

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/FathirSyahragilYusuf707012500050>